

RESEARCH

Open Access



An effective approach for classification of plant disease of small size dataset by using sequential deep learning model with Min–Max Hue histogram based segmentation

Vijay Kumar Trivedi¹, Piyush Kumar Shukla², Anjana Pandey³, Noha Alduaiji⁴  and J. Shreyas^{5*}

*Correspondence:
shreyas.j@manipal.edu

¹ School of Computing Science and Engineering (SCOPE), VIT Bhopal University, Bhopal, Madhya Pradesh 466114, India

² Department of Computer Science & Engineering, University Institute of Technology, Rajiv Gandhi Proudhyogiki Vishwavidyalaya (State Technological University of Madhya Pradesh), Bhopal, Madhya Pradesh 462033, India

³ Department of Information Technology, University Institute of Technology, Rajiv Gandhi Proudhyogiki Vishwavidyalaya (State Technological University of Madhya Pradesh), Bhopal, Madhya Pradesh 462033, India

⁴ Department of Computer Science, College of Computer and Information Sciences, Majmaah University, 11952 Majmaah, Saudi Arabia

⁵ Department of Information Technology, Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal 576104, India

Abstract

Training deep learning models for plant disease classification on small size datasets with heterogeneous backgrounds remains challenging issue. Proposed study overcomes the major limitations and weaknesses of the existing contemporary methods. This study introduces a novel three-phase approach: (1) segmentation of disease regions using a Min–Max Hue Histogram-based technique to isolate the disease region with the most important data, (2) a lightweight, sequential deep learning model (PDC-Net) trained from scratch to classify the diseases and (3) a disease recovery module to provide recommendation to farmer. Unlike existing works, we avoid data augmentation and transfer learning to eliminate overfitting risks and domain mismatches. Additionally, we created the Plant Disease Small Dataset (PDSD)—a new, realistic subset of the PlantVillage dataset featuring images with varied environmental conditions and backgrounds. Experimental results show that our method achieves an average classification accuracy of 91%, which is better than popular CNN architectures (VGG19 and ResNet50). This complete system provides an accurate, effective, and convenient way for farmers to diagnose plant diseases and to receive recovery strategy, which can help promote the precision agriculture practices.

Keywords: Contrast stretching, RoF filter, GIF filter, Hue histogram, HSI color model, K-mean clustering, Segmentation

Introduction

To maximize the crop yield, Indian farmers try to protect their crops from disease by regularly checking the crops and spotting disease at an early stage [1]. However, as simple as it may sound, the detection of plant diseases is not a simple task. Identification of these diseases requires specialized knowledge, which farmers usually lack. As a result, the farmers would need to consult specialists who can help them to identify plant illnesses, which may be fairly expensive, time-consuming, and the results are frequently unreliable. For the reasons outlined above, extensive study has been carried out to develop techniques that will be sufficient in accuracy and available to the majority of farmers without consulting specialists.

Various studies have been carried out in the context of automatic identification of plant diseases using a variety of techniques, including image processing, machine learning (ML), deep learning (DL), etc. Deep learning, particularly convolutional neural networks (CNN), provides more accurate categorization and diagnosis of plant disease results than machine learning, and allowing us to make better decisions.

Although the aforementioned studies have made a substantial contribution to the field of automatic disease diagnosis, only a small number of them have addressed the problem of data scarcity, which has an impact on the effectiveness of DL-based architectures. To address the challenge of small-sized datasets, current deep learning techniques for plant disease diagnosis often rely on data augmentation or transfer learning. However, these methods present limitations, such as the risk of overfitting due to overly similar augmented images or the potential for negative transfer effect in transfer learning [2]. Furthermore, most existing models inadequately address the critical need for precise disease spot segmentation, which is essential for accurate classification.

This paper addresses the practical challenges inherent in current precision farming strategies by proposing a novel approach designed to overcome identified limitations and empower farmers with enhanced decision-making capabilities for improved crop health. To address these critical research gaps, this study makes the following major contributions:

- We introduce a novel three-phase framework that includes (1) Min–Max Hue Histogram-based segmentation, (2) a custom sequential deep learning model (PDCNet) trained from scratch, and (3) a disease recovery recommendation module.
- We created a new subset of the PlantVillage dataset called the Plant Disease Small Dataset (PDSD), focusing on images with complex backgrounds and diverse environmental conditions, which better simulate real-world scenarios.
- Our approach demonstrates superior performance over state-of-the-art methods, particularly for small size datasets, by segmenting only the relevant disease regions and ignoring irrelevant background information.

Thus, this work fills important gaps in the identification of plant diseases on a small scale and provides a reliable, affordable, and useful recommendation system for precision farming.

The rest of the paper is structured as follows: Sect. "[Related works](#)" deals with recent work done in the field of disease classification with deep learning. Sect. "[CNN-based disease detection](#)" discusses a proposed working model of PDCNet (Plant Disease Convolutional Network) for detecting leaf disease using deep neural networks. Sect. "[Hybrid and ensemble models](#)" deals with experimental results and discussion. Summary and remarks are described in Sect. "[Pretrained and transfer learning approaches](#)".

Related works

Automatic classification of plant diseases has advanced significantly in recent years, largely driven by the adoption of deep learning techniques. This literature review provides a comprehensive overview of past studies, highlighting their favorable results and persistent challenges.

CNN-based disease detection

Initial studies, such as Mohanty et al. (2016), pioneered the use of CNN architectures like AlexNet and GoogleNet on the PlantVillage dataset, achieving a remarkable 99.35% accuracy across 38 plant disease classes [3]. Building on this, Toot et al. (2019) advanced the work by utilizing Inception-V3 for disease-specific classification, including bacterial spot and healthy leaves, with their model reaching an impressive 99.76% accuracy [4]. Karlekar et al. (2020) presented a two-stage architecture, SoyNet, which combined CNN classification modules with image processing modules (IPM). Their method effectively isolated leaf regions by subtracting complex backgrounds, leading to improved classification accuracy [5]. Kumart et al. (2021) further reinforced the efficacy of CNNs for plant disease detection across varied agricultural conditions [6].

However, challenges remain. So, Panchal et al. (2021) proposed a CNN-based model, trained on a publicly available dataset of 87,000 RGB images, which achieved 93.5% recognition accuracy. Despite this, their model suffered from class misclassification due to dataset limitations in size and diversity, resulting in decreased performance in real-world applications [7]. Similarly, Nkvetchakult et al. (2021) introduced a loss-fused, resilient CNN model tested on the PlantVillage benchmark dataset. While it attained 98.93% accuracy in controlled settings, it failed to maintain performance under diverse real-world conditions [8]. Pradeep et al. (2022) suggested a CNN-based EfficientNet model for multi-label and multi-class plant disease classification, utilizing deep feature extraction via hidden layers; however, its performance on benchmark datasets was not sufficient [9]. More recently, Tashin Ahmed et al. (2023) introduced a two-phase CNN architecture for rice crop disease detection. Their model, utilizing a limited dataset of only 200 images across three classes, efficiently identified regions of interest in the initial phase and classified them in the subsequent phase, demonstrating promising results despite the small dataset size [10].

Despite significant progress in this area, several persistent challenges need to be addressed. Many existing datasets are collected under controlled conditions, failing to reflect real-world variability, which often leads to models struggling with generalization in diverse field conditions. Moreover, these models frequently demand substantial computational power, and their results can suffer from overfitting or inadequate generalization to actual field conditions.

Hybrid and ensemble models

A CNN-SVM hybrid model was presented by Narayanant et al. (2022) for the categorization of banana leaves. The method used a CNN, and a fusion support vector machine (SVM) based classification system. Their approach utilized median filtering for preprocessing. Their accuracy was 99. However, the methodology lacked classification diversity [11]. Astan et al. (2022) proposed ensemble classifiers which evaluated using the PlantVillage and Taiwan Tomato Leaves Datasets. Their experiments demonstrated promising results [12].

Pretrained and transfer learning approaches

Chent al. (2020) used a transfer learning technique with ResNet50 to classify plant diseases. Their result achieved an accuracy rate of 98.9% [13]. Jadhav et al. (2020) proposed transfer learning-based plant disease detection system which utilized AlexNet and GoogleNet for classification [14]. In addition to these, Kabir et al. (2020) proposed a multi-label CNN model for classifying various plant diseases using transfer learning techniques [15], including AlexNet [16], GoogleNet [17], VGG19 [18], ResNet50 [19], ImageNet [20], Dense121 [21], LeNet [22], and XceptionNet [23]. The authors claimed theirs was the first study to use a multi-label CNN to classify 28 different plant disease subtypes. Although effective in specific contexts, these models showed reduced performance when applied to more diverse, real-world datasets. Nkvetchakul and Surinta (2022) introduced a CNN-based methodology that used transfer learning to categorize two particular plant diseases. The NAS MobileNet and MobileNetV2 networks were employed in their model. In their experiments, NAS MobileNet gave them more accurate predictions. A number of different data augmentation procedures, including rotation, zooming, brightness alteration, and mixing, were used by them in order to mitigate the issue of overfitting. They achieved a maximal test accuracy of 84.51% by utilizing the Cassava 2019 a separately collected leaf disease dataset [24].

To address the issue of data scarcity, Arsenovic et al. (2019) expanded their research by creating a large dataset of plant images captured under varying weather conditions, angles, and daylight exposures. They also proposed a GAN-based augmentation method combined with a two-stage detection algorithm tailored for real-time environments [25]. In addition, Abbasi et al. (2021) and Anht et al. (2021) employed pre-trained CNNs like MobileNet and generated synthetic data using GANs [26, 27].

Even though many researchers use data augmentation, it often creates duplicate images, which can lead to overfitting (Cogswell et al., 2015). Negative transfer is also a barrier for transfer learning when the source and destination domains are significantly different [28].

Recent research has advanced the field of plant disease detection by paying more attention to small datasets, lightweight models, and sophisticated learning techniques. For example, PlantXViT (2022) achieved more than 93% accuracy by combining CNN and Vision Transformer (ViT) components together. With just 0.8 M parameters, they achieved good results [29]. MSNet (2023) combined EfficientNet and DenseNet with attention modules, in order to achieve good results while using very few resources [30]. Moreover, transformer topologies can achieve high accuracy even with minimal training data, as demonstrated by Zhao et al. (2022), who presented a novel transformer-based model for the purpose of plant disease detection [31]. While the newly introduced CNN network architecture with attention mechanisms by Ahmed et al. (2023) was able to reach a high level of accuracy in the identification of tomato diseases in authentic field images [32], Yang et al. (2024) looked into lightweight CNN architectures which were designed to operate efficiently on low-power peripheral devices, thereby addressing the challenge of deploying plant disease classifiers in resource-limited environments [33]. Lastly, Wang et al. (2024) proposed a self-supervised contrastive learning method which greatly improves the classification performance on small, mixed plant disease datasets [34].

These recent advancements indicate a growing trend toward utilizing compact and efficient models and self-supervised learning to address the challenge of limited data in plant disease detection. This direction aligns well with our approach, which integrates ROI-based segmentation with a shallow deep learning model to achieve efficient and accurate solutions.

The PDCNet model mitigates the prevalent limitations observed in earlier plant disease classification models through the following key advancements:

- Reduces digital noise, enhancing visualization of disease spots.
- Does not require data augmentation or transfer learning.
- Identify disease areas prior to segmentation utilizing the hue histogram method.
- More precise segmentation with the help of Min–Max Hue Histogram and K-means clustering
- No GPU is needed for the proposed lightweight architecture.
- Achieved 91.25% accuracy, demonstrating superior performance on small, heterogeneous datasets.
- Developed a recovery recommendation system for farmers.

Proposed methodology

To enhance learning efficiency from a limited collection of plant images, particularly those with significant background heterogeneity, and to provide the farmers with an actionable solution, the proposed system is logically divided into the following modules:

1. Leaf Segmentation Module (LSM).
2. Deep Learning Module (DLM).
3. Disease Recovery Module (DRM).

Figure 1 presents a comprehensive perspective of the proposed system. It demonstrated the proposed disease recognition and recovery procedures in a step-by-step manner:

If the CNN architecture is merely trained to locate the significant part only, it may perform better results even with a small size dataset. As a result, the first module works with segmenting plant images in order to extract the major disease regions from the image. The original image is taken, resized to a predetermined size, and then sent to the pre-processing step, where Rank Order Fuzzy (RoF) filter and GIF (Guided Image Filtering) filter techniques are performed to remove undesirable noise and boost image contrast, respectively. On these improved images, we apply our proposed Min–Max Hue Histogram-based disease spot detection technique, in which image pixels with hue values between the min and max values of the hue histogram are combined with some threshold value to separate the disease regions from the leaves. Following the identification of important disease regions from an image, these regions are extracted using k-mean clustering algorithms. These images appear simply due to the absence of a heterogeneous background. Following that, these simplified images are fed into the proposed PDCNet architecture, which is trained with this simplified dataset to recognize disease.

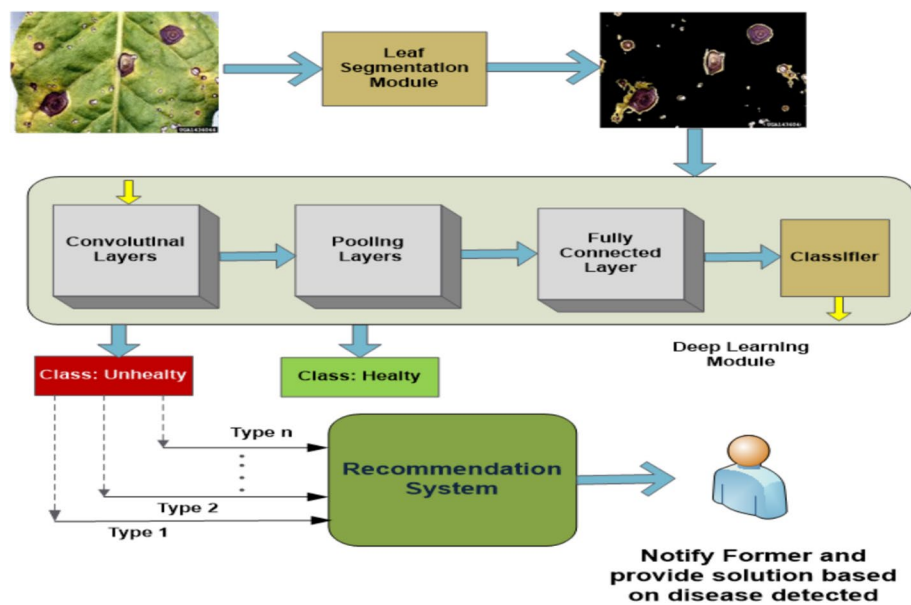


Fig. 1 Proposed working model

experiments have demonstrated that the learning process is simple and effective. Finally, the disease recovery phase is carried out, where system can advise farmers on the disease and how to recover from it.

Detailed explanations of each module are provided in the following subsections:

Leaf segmentation module

In almost all of the images in the plant village database [35], it has been found that a portion of the leaf's surface is significantly lower than the portion of the background, and the amount of diseased tissue that can be found on a leaf is extremely negligible in comparison to the whole background area. Therefore, the classification accuracy may be affected if CNN is trained without background removal. Alternatively, we can say that, before training the CNN, if the leaf disease portions of the image are separated from the entire image, it will be quite simple for the CNN to find and learn important features, even if there are only a limited number of images available for training. Therefore, it is essential to perform background subtraction before training while the infected areas are still small. Here, we have applied LSM to completely remove the background from the image. LSM is made up of four sub-modules, and their results are shown in Fig. 2.

Image acquisition and preparation

In image acquisition, a leaf image is obtained and serves as the designated image for subsequent processing. In order to acquire and prepare image data, we used a portion of the PlantVillage dataset [35] that included 12 different types of healthy and infected leaves. This dataset comprises 3598 images of plant leaves with their respective labels. It includes a collection of images taken in different conditions.

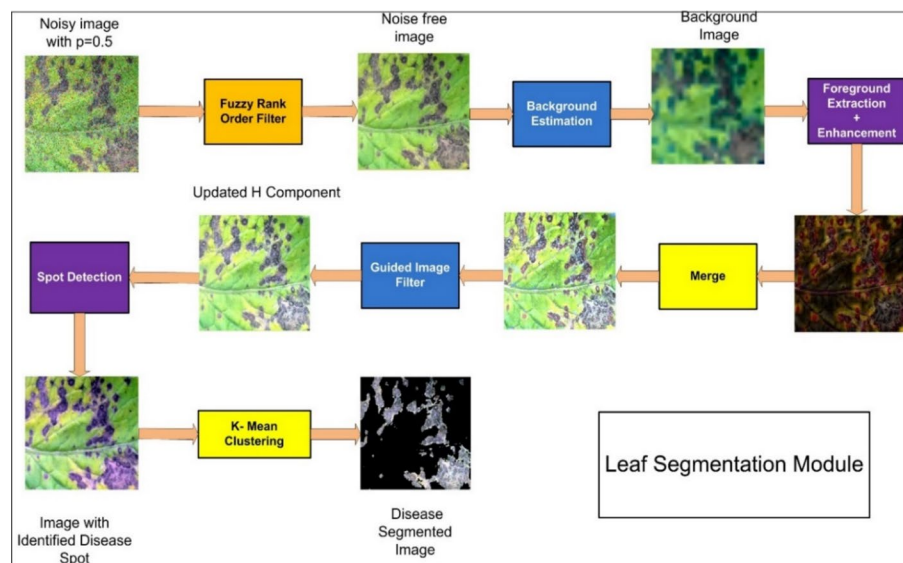


Fig. 2 Working of Leaf Segmentation Module

Pre-processing

Pre-processing of the input image is an essential step to improve the quality of the image and eliminate undesired distortions. The pre-processing step consists of a combination of image denoising and foreground enhancement methods that produce a higher-quality image. For image denoising, the RoF filter [36, 37] is used. Image improvement with the RoF filter is accomplished by the use of fuzzification, which transfers images from the spatial domain to the fuzzy domain, enhancement of fuzzy membership values, and defuzzification, which transforms the original image back to the spatial domain. The RoF filter is used to enhance the quality of extremely noisy images, particularly in cases where the noise level is highly distorted. RoF uses the ranking/density of noisy images as a fundamental step for noise detection. Next, to extract foreground objects from the denoised image, we suggested a novel background subtraction method that employs a local pixel homogeneity scheme from CIE $L^*a^*b^*$ colour space [38] to estimate the background and a local intensity pattern comparison from the RGB colour model to detect the foreground. After detecting the foreground objects, these objects are enhanced by using an adaptive image improvement method called the GIF filter [39]. Based on the distribution features of the light intensity in a crop image, the GIF approach uses Otsu's thresholding [40] to automatically generate enhancement weights and adaptively change its contrast.

Leaf disease spot identification

Leaf disease spot identification is the third step of the leaf segmentation module. In this step, improved leaf images are processed by using the min-max hue histogram-based technique to identify an interested image area. Initially, the RGB picture undergoes a transformation to become an HSI color image. The H component adjustment is performed to provide a threshold value that distinguishes disease spots from plant

leaves. In order to do this, calculate the histogram of the hue component and identify the greatest and lowest peak points between 0 and 60°, in the histogram that correlate to the color associated with the illness [41]. These two points, designated $\tau 1$ and $\tau 2$, are used as thresholds.

Next, we use the following formula to find and isolate the illness spot inside the picture of the leaf.

$$H_{ij}' = \begin{cases} H_{ij} & : H_{ij} < \tau 1 \\ H_{ij} + \eta & : \tau 1 \leq H_{ij} < \tau 2 \\ H_{ij} & : H_{ij} \geq \tau 2 \end{cases} \quad (1)$$

where the hue values of the input and output image are represented as H_{ij} and H_{ij}' respectively. η is another threshold that amplifies the hue value in the image in order to distinguish the diseases.

Following that, contrast stretching [42] technique is applied to the I component, which then results in an increase in the intensity factor globally.

Finally, combine the modified H and I components with the unchanged S component to get the final improved HSI color picture. This final improved HSI color map is then, converted back into the RGB color map that shows the image with illness spot identification [43].

Leaf segmentation

In the leaf segmentation step, the image is partitioned into various clusters by using K-mean clustering [44]. The K-mean clustering is applied on the L*a*b* color model [38]. After applying, the K-means clustering method, all of the clusters are then translated from the RGB color model to the HSI color model. A cluster is considered to be a disease cluster if it has a bigger hue mean value than the other clusters.

After getting the important part of an image, the smoothing filter is used to make the image look better. Because of the lack of a varied background, these images would appear simplistic.

Deep learning module

Pre-processing and segmentation techniques are used to separate the important area from the plant leaf that is the region of interest. The results from the LSM module are then put into classes so that identification and decisions can be made. The method known as Plant Disease Convolutional NETwork (PDCNet) is used for the classification process. In this step, the relevant information is obtained to classify the various crop disease types.

One of the branches of machine learning is known as "deep learning." Because it has numerous layers, a deep convolutional neural network (DCNN) is able to gradually extract higher level information from the images given as input. VGG19, ResNet50, GoogLeNet,tc. are some of the most widely used models for classifying images. These models perform poorly on datasets that are too specific. These networks can learn image information well enough to classify images into a wide range of categories. When the differences between the visuals are not particularly noticeable, they struggle to learn.

In order to categorize plant diseases using limited datasets, the suggested network, which is referred to as PDCNet, was developed specifically for this purpose. Due to the fact that it is able to learn the more complicated properties of the dataset, it is able to accurately identify illnesses. The structure is defined in such a manner that each following layer is able to extract progressively complicated properties from the leaf data. The spatial resolution of an image is decreased with each successive layer, which has made it possible for us to see fine details of the leaf.

This section describes in detail the description of the structure of PDCNet, which consists of the convolutional layer, activation functions that is rectified linear unit (ReLU) and softmax, the pooling layer, and the dropout layer. The subsequent subsections will present the detailed setup of each layer that makes up PDCNet.

Model architecture

Figure 3 displays a visual illustration of the proposed PDCNet model. The PDCNet primarily consists of two fully connected layers F1 and F2, one flatten layer, five max-pooling layers (P1, P2, P3, ..., and P5), and five convolutional layers (C1, C2, C3, ..., and C5). The convolutional layers are primarily responsible for improving the model's representational ability. The maxpool layer has 2×2 filters, while each convolution network has a 11×11 filter. The architectures of PDCNet to support the different defined classes shown in Table 1.

Convolution layers

Convolution, which is the backbone of CNNs, is characterized as an operation on two functions. CNNs convolute the feature map with multiple inputs and pass the result to an activation function. Convolution in CNNs is expressed as follows:

$$C_i = \Psi(I * Z_i) \quad (2)$$

The convolution operation, denoted as $*$, is applied to the extracted leaf image, I , using a kernel or mask Z_i in the i th layer. The resulting convoluted leaf image is then sent through the activation function $\Psi(\cdot)$. The PDCNet consists of five convolutional

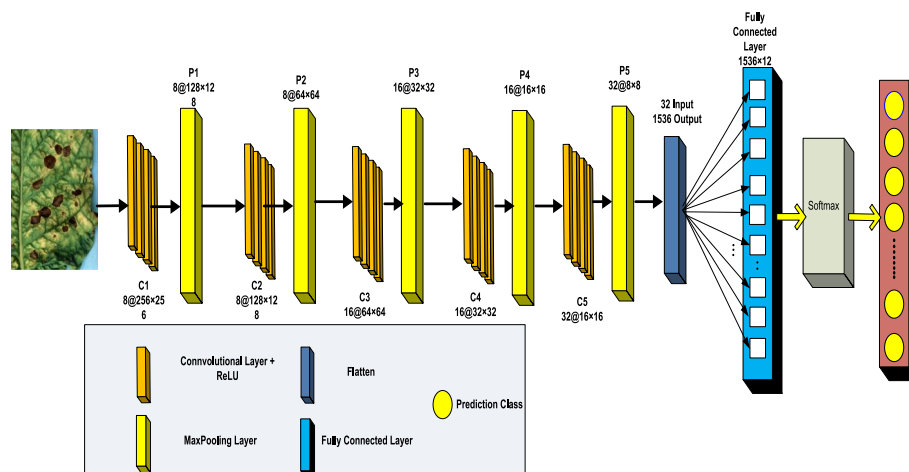


Fig. 3 Architecture of PDCNet

Table 1 Layers and its description of proposed pdcnet model

Function Name	Layer	Filter Size	Stride	Output tensor
Convolutional + ReLU	Conv1	11 × 11 × 8	1	8 × 256 × 256
Max Pooling	Pool1	2 × 2	2	8 × 128 × 128
Convolutional + ReLU	Conv2	11 × 11 × 8	1	8 × 128 × 128
Max Pooling	Pool2	2 × 2	2	8 × 64 × 64
Convolutional + ReLU	Conv3	11 × 11 × 16	1	16 × 64 × 64
Max Pooling	Pool3	2 × 2	2	16 × 32 × 32
Convolutional + ReLU	Conv4	11 × 11 × 16	1	16 × 32 × 32
Max Pooling	Pool4	2 × 2	2	16 × 16 × 16
Convolutional + ReLU	Conv5	11 × 11 × 32	1	32 × 16 × 16
Max Pooling	Pool5	2 × 2	2	32 × 8 × 8
Dropout	flatten	--	-	1536 × 1 × 1
SoftMax Regression	Output	--	-	12 × 1 × 1

layers,achquipped with an 11 × 11 filter. While there are only 8 neurons in the first two convolution layers, there are 16 neurons in the following two layers. The final convolution layers include 32 neurons.

Activation function

Rectified linear unit During the training phase, ReLU activation is chosen in all convolution network layers due to its superior performance in training over other activation functions like sigmoid and tanh. ReLU produces x as an output if the input, x, is positive;lse, it produces 0. The following is a mathematical definition of the ReLU function:

$$\text{ReLU}(v) = \begin{cases} v & \text{if } v \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

It is often referred to as the ramp function. It requires the input C_i , which is derived by applyingq. (2).

Softmax The softmax activation is used in the final dense layer to calculate the probability of predicted classes. The output is chosen to be the class with the highest probability. The formula for the softmax function is:

$$S(y_i) = \frac{e^{y_i}}{\sum_{j=1}^k e^{y_j}} \quad (4)$$

In this work, the variables e^{y_i} and e^{y_j} represents the likelihood of belonging to the categories i and j respectively. The variable K represents the number of categories and it is initially set to twelve in this work. The prediction forach class is determined using the softmax function $S(y_i)$.

Pooling layer

Pooling layer is responsible for lowering the overall output of the convolutional layer, and, as a result, lowering the number of trainable parameters. When a large number of convolutional layers are used, the training parameters also grow so quickly. Pooling layers are used to solve this issue. The pooling layer uses a region's overall spatial characteristic to represent the entire region, result in reducing the convolutional layer's output and trainable parameters. The PDCNet uses max pooling with a 2×2 window size. The maximum value that is present in the 2×2 window is what the max pooling operation reflects.

Loss function

The loss function is responsible for measuring the disparity between the output that was anticipated and the output that was wanted (the label value). PDCNet uses categorical cross-entropy to calculate loss. The following is the categorical cross-entropy function:

$$L = - \sum_{j=1}^k \frac{y_j \log(\hat{y}_j)}{K} \quad (5)$$

where y and $\hat{y}$ represent the anticipated and the desired outputs respectively.

Optimization and weight updation

The primary objective in the process of training a neural network is to determine the optimal value of a parameter, denoted as θ , that minimizes the loss function L . In order to facilitate expedited convergence during training, the model utilizes the Adaptive Moment Estimation (ADAM) technique [45] for optimization and weight updates. In this work, Adam [45] employs a parameter update technique that bears resemblance to Root Mean Square Propagation (RMSP) [46], with the inclusion of a momentum term. The parameter gradients and their squared values are kept in an element-wise moving average. Following are the criteria for updating the weights:

$$w_{\ell+1} = w_{\ell} - \alpha \frac{m_{\ell}}{\sqrt{v_{\ell} + \epsilon}} \quad (6)$$

where,

$$m_{\ell} = \beta_1 m_{\ell-1} + (1 - \beta_1) \left[\frac{\delta L}{\delta w_{\ell}} \right] \quad (7)$$

$$v_{\ell} = \beta_2 v_{\ell-1} + (1 - \beta_2) \left[\frac{\delta L}{\delta w_{\ell}} \right]^2 \quad (8)$$

w_{ℓ} = weights at time ℓ .

$w_{\ell+1}$ = weights at time $\ell + 1$.

α_{ℓ} = learning rate at time ℓ .

δL = derivative of Loss Function.

δw_ℓ = derivative of weights at time ℓ .

v_ℓ = sum of square of past gradients. [i.e. $\sum (\partial L / \partial W_t - 1)$] (initially, $v_\ell = 0$).

β = Moving average parameter (const, 0.9).

ϵ = A small positive constant (10^{-8}).

Dropout

To prevent the network from overfitting and enhance its generalization capabilities, a dropout layer was included into the structure. With a certain probability, the dropout layer randomly sets all of the input elements to zero. During the training phase, the dropout mask supplies the input components, and the layer resets them to zero in an unpredictable manner.

$$\text{rand}(\text{size}(X)) = P(X) \quad (9)$$

Here, X represents the layer input, which is subsequently multiplied by the scaling factor of $1/(1-P)$ to get the remaining components. In this layer, no learning occurs, similar to the max or average pooling layers.

Disease recovery module

In the disease recovery phase, post-diagnosis recommendations are generated. Based on the identified leaf disease, the system provides tailored solutions for remediation. For example, for anthracnose leaf disease—characterized by symptoms such as yellowing leaf tips turning brown and association with seed/plant debris—the system would recommend specific recovery actions like removing infected leaves and avoiding overwatering.

Experimental results and discussion

Dataset

For the experimental investigation, we took into consideration the standard, open-access PlantVillage dataset [35]. This dataset is made up of 55,328 different images of healthy plant leaves as well as images of infected plant leaves, and it was collected from fourteen different types of plants, such as tomato, blueberry, cherry, maize, grape, orange, peach, pepper, potato, raspberry, squash, and strawberry. The digital images of 14 different types of crops were divided into 38 categories. Within these 38 classes, there are images of sickness in 26 of the classes, while there are healthy images in 12 of the classes. In order to make the plant disease detection model more applicable for use in the real world on tiny datasets, a new subset of this dataset has been created and given the name Plant Disease Small Dataset (PDSD). This dataset contains only those images that look different from others, such as irrelevant objects, different ground textures, different scaling, different shading effects, etc. Approximately 15% of images are taken from each class. Each image includes only one disease with complex background. The PDSD is divided into 12 sub-categories, including healthy leaves, which are Bacterial_blight, Black_root, Black_measles, Cercospora, Isariopsis_leaf_spot, Late_blight, Leaf_scorch, Mosaic_virus, Northern_blight, Powdery_mildew, Rust, Septoria/Brown_leaf_Spot. Some sample images along with their diseases' name are shown in Fig. 4. About 80% of the images from each class will be allocated for training the classifier, while the remaining 20% will be utilized for testing. The samples per class of the dataset is summarized in Table 2.

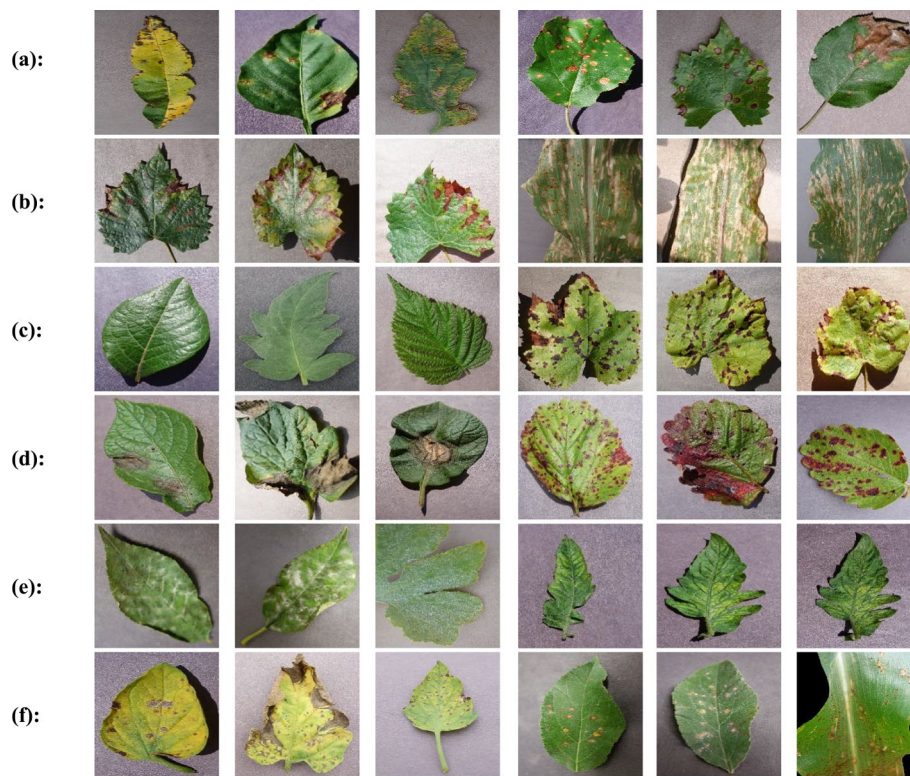


Fig. 4 some leaf photos of PSDS database: **a** Bacterial blight and Black root, **b** Black_Measles and Cercospora **c** Healthy and Isariopsis_Leaf_Spot **d** Late blight and Leaf Scorch Powdery_mildew and mosaic_virus **f** Septoria/Brown Leaf Spot and Rust

Table 2 Statistics of plant disease small dataset

Class Name	Disease Name	Type of plant	# Images
D-1	Bacterial_spot	Papper, Peach, Tomato	360
D-2	Black_root	Apple, Graps	478
D-3	Black_Measles	Grape	240
D-4	Cercospora	Corn	172
D-5	Healthy	All	580
D-6	Isariopsis_leaf_spot	Grapes	225
D-7	Late_blight	Potato, Tomato	350
D-8	Leaf_corch	Strawberry	215
D-9	Powdery_mildew	Cherry, Squash	340
D-10	Septoria/Brown_leaf_spot	Tomato	308
D-11	mosaic_virus	Tomato	200
D-12	rust	Apple, Corn	130

Valuation matrices

Accuracy is one of the most often used valuation measure. It describes how closely a value predicted by a machine algorithm equals to its actual value. Accuracy is directly proportional to efficiency; hence, a greater value is preferable. However, due to the accuracy paradox, depending entirely on accuracy can sometimes lead to incorrect conclusions.

Therefore, to evaluate machine learning algorithms, accuracy is combined with additional performance indicators such as precision, recall, and f1-score. The confusion matrix impacts all of the metrics mentioned above. Here, we use confusion matrix to evaluate the proposed method in comparison to state-of-the-art methods.

Tables 3 and 4 show the confusion matrix for two classes as well as performance measures, respectively.

Recall quantifies the percentage of positive test results from all positive samples. The FPR measures the percentage of false positives with respect to available negative samples. ACC and F-measure are crucial for assessing binary classification quality. The harmonic mean of "precision" and "recall" is used to get the F-measure, and it is defined as:

$$F - measure = \frac{2 \times recall \times precision}{recall + precision} \quad (10)$$

System configuration

Model training and testing are done using the MATLAB programming language. The basic configuration of the system is presented in Table 5. Because the PDCNet is being used to classify leaf diseases, it was decided to train the network from scratch. The PDCNet differs from the other networks in this study due to its training technique.

Table 3 Confusion matrix

Actual Class	Predicted class	
	Positive	Negative
Positive	TP (true positive)	FN (false negative)
Negative	FP (false positive)	TN (true negative)

Table 4 Performance parameter

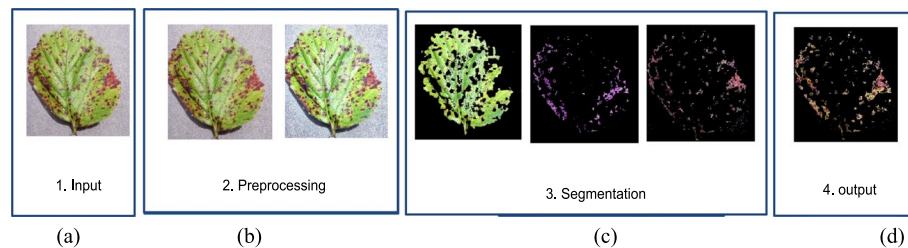
Measure	Definition
TPR or recall	TP/ (TP + FN)
FPR	FP/ (FP + TN)
precision	TP/ (TP + FP)
ACC	(TP + TN/K)

Table 5 Configurations of machine

Name	Parameters
Memory	16 GM
Processor	12th Gen Intel(R) Core (TM) i5-1240P 1.70 GHz
Graphics	Intel(R) Iris(R) Xe Graphics
Programming Language	MATLAB R2022b
Operating System	Windows 11 version 22H2

Table 6 Hyper parameters used for training pdcnet

Name	Parameters
Solver Name	ADAM
Learning-Rate	3e-4
Hardware Resource	Single CPU
Gradient Threshold Method	l2norm
L2 Regularization	0.01
Learn Rate Drop Factor	0.25
Gradient Decay Factor	0.9900
Batch Size	64
Maxpochs	25

**Fig. 5** Segmentation and extraction Steps: **(a)** Input Images, **(b)** Images Applied Preprocessing, **(c)** Images with Segmented Regions **(d)** ROI extracted

Hyper parameters

The hyperparameters that were utilized to train the model are listed in Table 6. The ADAM solver type is used for this model. The initial learning rate is set to 0.001. The training and testing sets are divided into multiple batches using batch training. Each batch has 64 images in it. The maximum epoch is set to 25. L2 regularization is also adopted to minimize the overfitting of the model, and it is set to 0.01.

Segmentation accuracy

The majority of the images in the PDS dataset contains a heterogeneous background. So, it is a challenging task to employ any CNN architecture in a straightforward way to produce effective results because of this heterogeneity. This challenging task has been broken down into two straightforward tasks in the case of our suggested technique. The first task is to identify the important area (ROI) from the image, and then the second task is to perform the segmentation of diseases with the help of this identified important area, which lacks any major background elements. As a result, the suggested strategy was successful in producing good results on a small dataset gathered from a plant village dataset.

Figure 5 depicts the disease segmentation and extraction procedures. The output of these procedures is simplified images with no or very little heterogeneity. These images are transmitted to the architecture's dense layers; each image must have a fixed dimension of 256 by 256 pixels. Each Region of Interest (ROI) image is processed through several convolutional and pooling layers, finally getting output as a 32 × 8 × 8 image. A dropout layer was added to the network in order to prevent overfitting and increase generalization.

PDCNet contains fully connected, dense layers. All of the feature maps that are $1536 \times 1 \times 1$ in size are attenuated and passed through these completely connected layers. The final layer includes twelve branches, and as illustrated in Fig. 6, each branch of the CNN model predicts whether a grain class is healthy or diseased.

Figure 7 illustrates the outcomes of segmenting leaf disease images using hue-based spot detection and K-means clustering. Each picture consists of four columns: the first column displays the original input image, the second column shows the image after pre-processing, the third column displays the diseased region discovered using Hue-based spot detection, and the fourth column shows the diseased section segmented using the K-means clustering method.

DCNN performance evaluation

Impact of leaf segmentation phase

This section evaluates the performance of the final PDCNet model on a customized dataset (a subset of the PlantVillage dataset) and assesses the impact of the proposed leaf segmentation step on its overall efficacy. For this purpose, two distinct experiments were conducted.

In the first experiment, the performance of the model was assessed using simplified images of a PDS dataset that contained only significant disease areas without a heterogeneous background. For that purpose, CNN architecture was trained using the training parameters shown in Table 6. It was trained with these simplified images of PDS and tested on the same PDS dataset. In the PDS (12 classes, 3598 images), the training set was allocated 80% of the data, while the remaining 20% was assigned to the test set.

Figure 8 shows a training set loss decreasing with each iteration. The loss value is successfully decreased as the model gains experience, and it displays proper convergence toward the global minima. Around the 21st epoch, the training losses achieve their minimal levels, and after that, the loss values have mostly remained stable. Figure 8 further shows how training set accuracy improves with each iteration. Table 7 displays the lowest loss and maximum accuracy values achieved for the training as well as test sets.

Figure 9 presents the confusion matrix, which provides a comparison between the target class and the expected class. In this study, we evaluated the performance of various diseases using parameters precision and recall, as well as F1-score metrics by analyzing the confusion matrix, as depicted in Fig. 9. The average-precision, average-recall, and average-F1-score is found to be 0.91, 0.91, and 0.9092 respectively. The class-wise performance for each and every disease in the test sets can be found in Table 8.

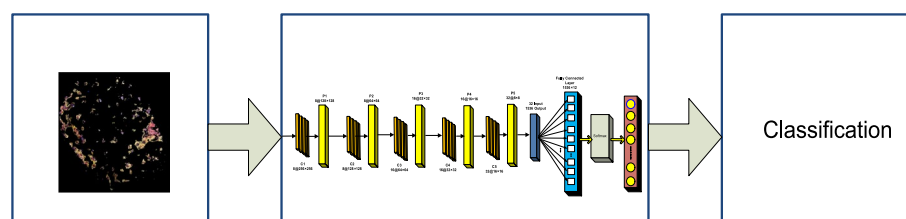


Fig. 6 Process of Classifying

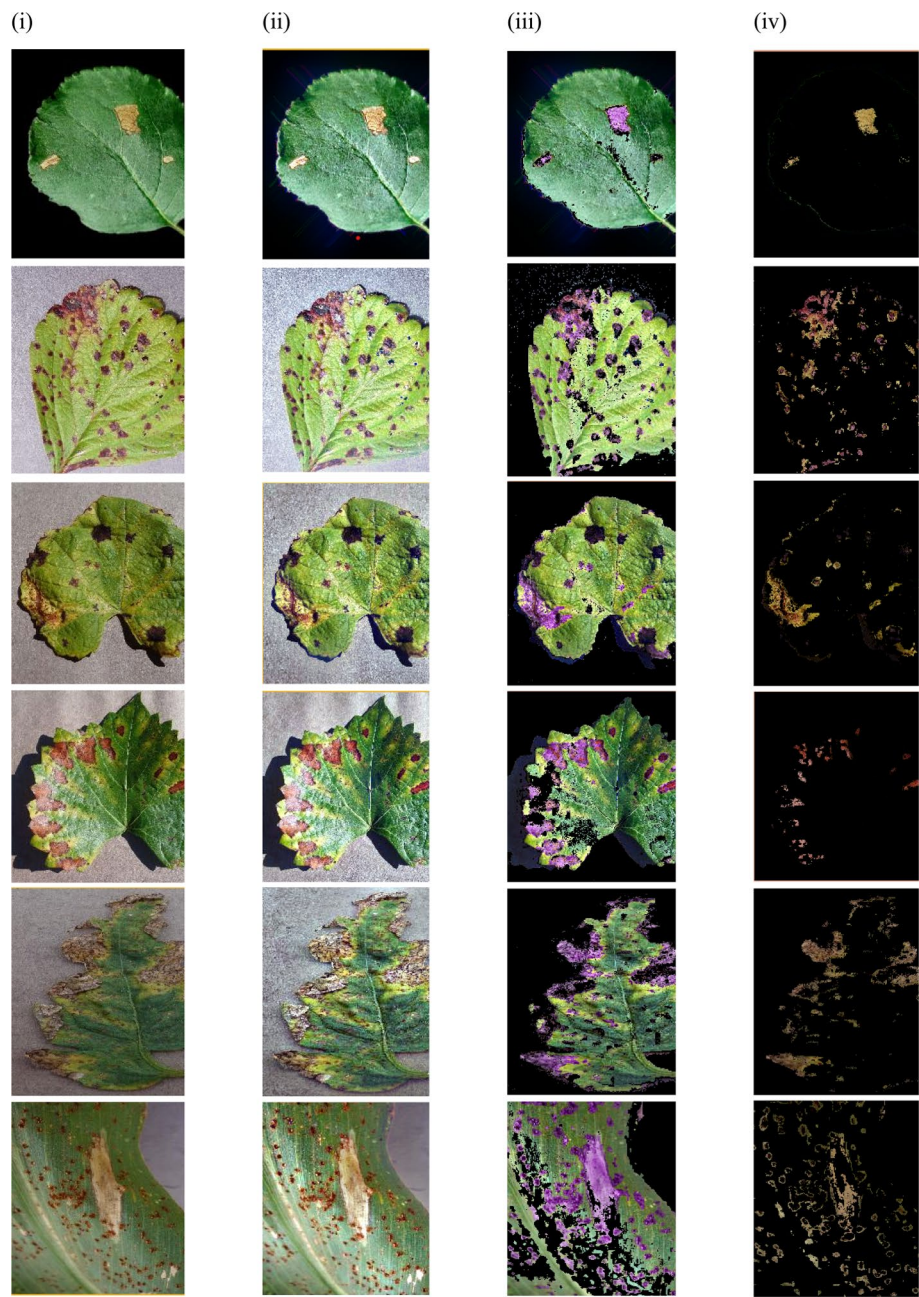


Fig. 7 Some examples of the output of Min–max Hue histogram and k-mean clustering-based segmentation (ii) Noise Removed image (iii) spot detection (iv) Segmented image using K-Mean Clustering

The second approach included the same CNN architecture and hyperparameter values, but rather than putting the simplified segmented images, we put images directly from the PDSD dataset into the training, and test set. Figure 10 shows training progress consisting of accuracy and loss calculation for each iteration. Table 9 displays the top 1 result from the training and test phase of the second experiment.

Figure 11 presents the confusion matrix of the model performed on second experiment. The average-precision, average-recall, and average-f1-score which is drawn is

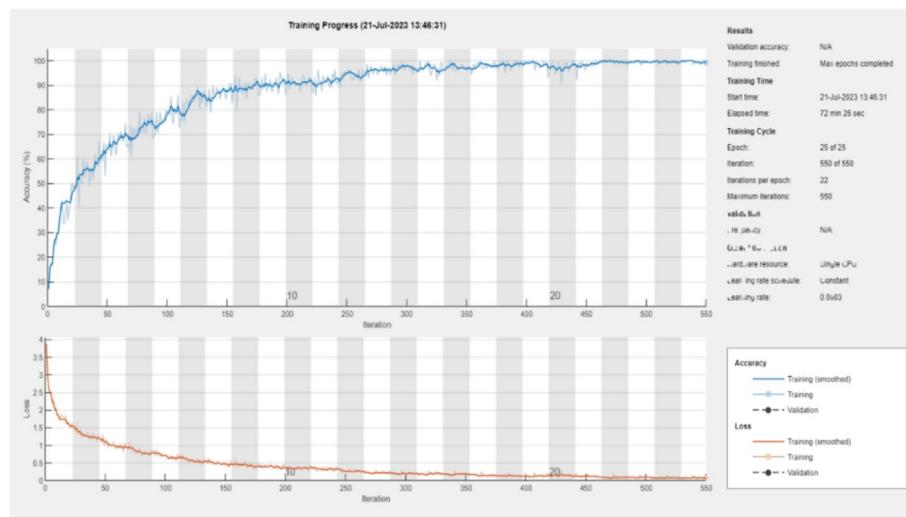


Fig. 8 Training Progress Graph of Proposed PDCNet for firstxperiment

Table 7 Loss and accuracy for training and testing sets on firstxperiment

Dataset	Loss	Accuracy
Training	0.095	0.9910
Testing	0.145	0.9125

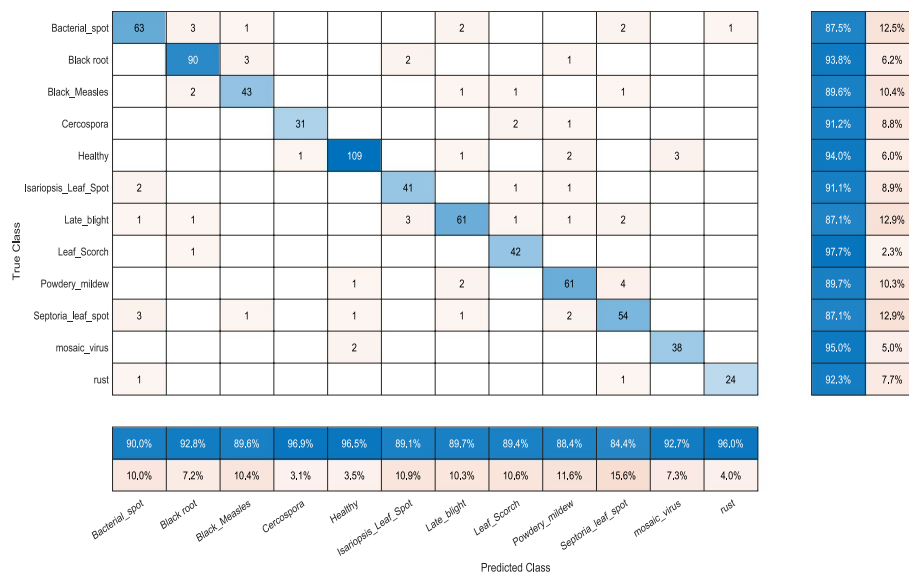


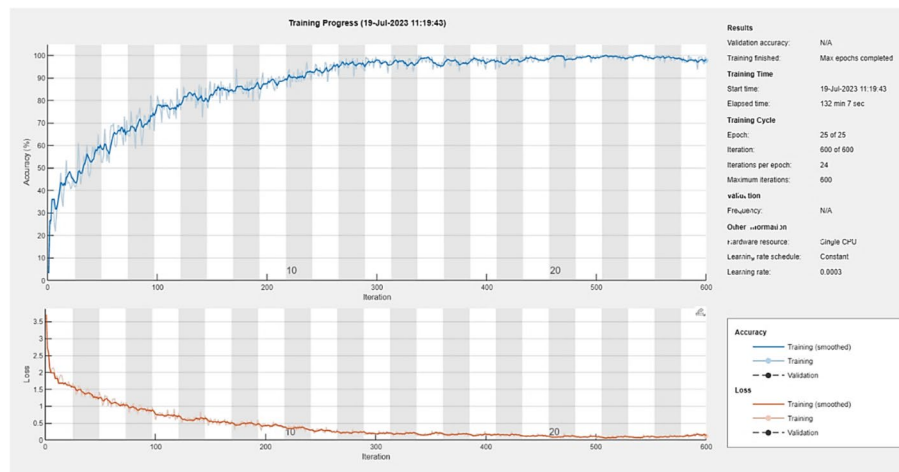
Fig. 9 confusion matrix of segmented PDS dataset

0.85, 0.84, and 0.8434 respectively. The class-wise performance for each and every disease on the second test sets can be found in Table 10.

Table 11 presents the classification accuracies achieved during model training on two distinct datasets, valuating performance across twelve classes. Two fundamental

Table 8 Precision, recall and F1-score for test dataset on firstxperiment

Disease Type	Precision	recall	F1-score
D-1	0.90	0.87	0.8847
D-2	0.92	0.93	0.9250
D-3	0.89	0.89	0.8900
D-4	0.97	0.91	0.9390
D-5	0.96	0.94	0.9499
D-6	0.89	0.91	0.8999
D-7	0.89	0.87	0.8799
D-8	0.89	0.97	0.9283
D-9	0.88	0.89	0.8850
D-10	0.84	0.87	0.8547
D-11	0.92	0.95	0.9348
D-12	0.96	0.92	0.9396
Avg	0.91	0.91	0.9092

**Fig. 10** Training Progress Graph of proposed PDCNet for secondxperiment**Table 9** Loss and accuracy for training and testing sets on secondxperiment

Dataset	Loss	Accuracy
Training	0.134	0.9856
Testing	0.168	0.8486

training and testing methodologies were applied: an 80/20 train/test split on the original PDS images, and the same split on simplified PDS images. The measurements shown include:

- The classification accuracy for both healthy and diseased categories withinach class.
- The average model loss, defined as the average loss per batch across all batches in both the training and testing sets.
- The trainingpoch at whichach model achieved its optimal performance.

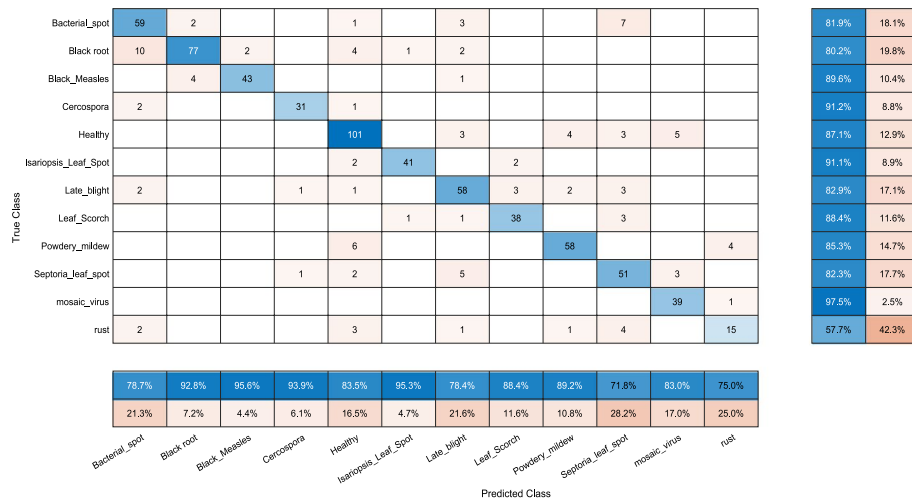


Fig. 11 confusion matrix of actual PSDS dataset

Table 10 Precision, recall and F1-score for test dataset on second experiment

Disease Type	Precision	recall	F1-score
D-1	0.78	0.81	0.7947
D-2	0.92	0.80	0.8558
D-3	0.95	0.89	0.9190
D-4	0.93	0.91	0.9199
D-5	0.83	0.87	0.8495
D-6	0.95	0.91	0.9296
D-7	0.78	0.82	0.7995
D-8	0.88	0.88	0.8800
D-9	0.89	0.85	0.8695
D-10	0.71	0.82	0.7610
D-11	0.83	0.97	0.8946
D-12	0.75	0.57	0.6477
Avg	0.85	0.84	0.8434

Table 11 Performance of proposed model on two different datasets

Name	Original images	Segmented images
Success Rate	85	91
Averagererror	0.15	0.1
Epoch	25	25
Time(m/epoch)	5.25	2.75

- The average time perepoch (in seconds) required for model training.

The results shown in Table 11 indicate that the suggested model shows superior results when employing segmented plant leaf images. Consequently, this strategy

requires less training time. The model achieved the highest success rates when applied to segmented images, as demonstrated in the result.

Comparison with existing state-of-the-art models

In order to thoroughly evaluate the performance of the proposed PDCNet model, we conducted experiments comparing it to several prominent CNN-based architectures, including VGG19, GoogleNet, ResNet50, and DenseNet, as well as transfer learning methods that leverage these pre-trained models. All these models were trained and evaluated on the newly created PDS dataset with an 80/20 split for training and testing.

Table 12 shows the comparative performance of each model in terms of accuracy, precision, recall, and F1-score. Although it is commonly observed that deeper networks such as VGG19 achieve better performance on large-scale datasets due to their ability to extract hierarchical features, our results (Table 12) show that the shallower PDCNet architecture outperformed VGG19, GoogleNet and ResNet on the PDS dataset. This can be attributed to several factors. First, the PDS dataset is relatively small and has a simpler feature space because of the explicit ROI-based segmentation we applied, which removes background noise and redundancy. As a result, very deep networks like VGG19 are more prone to overfitting in this scenario, whereas PDCNet—designed with a simpler architecture and fewer parameters—was better suited to generalize on this dataset. Second, the tailored segmentation steps ensure that the input to PDCNet already contains only the relevant features, reducing the need for extremely deep feature extraction layers. Finally, our PDCNet was trained from scratch on the dataset, avoiding negative transfer issues that can occur when fine-tuning deep networks pre-trained on other domains (like VGG19). Overall, these factors collectively contribute to PDCNet's superior performance despite its shallower architecture.

Figure 12 graphically summarizes the comparative results, illustrating the superior performance of the PDCNet model across all four metrics. The findings clearly demonstrate that the proposed PDCNet with segmentation-based preprocessing provides improved performance on small, heterogeneous datasets compared to these state-of-the-art CNN architectures. This validates the efficacy of our three-phase approach, particularly in scenarios with limited real-world data where data augmentation or transfer learning may not suffice.

Comparison with other existing models

The majority of previous work have used PlantVillage, which is based on combination of plant name and disease labels such as “Peach_Bacterialspot” vs. “Peach_Healthy”, to

Table 12 Comparative analysis with four state-of-art methods

Models	Accuracy	Precision	Recall	F1-Score
VGG19	0.89	0.90	0.88	0.8898
GoogleNet	0.64	0.68	0.71	0.6946
ResNet50	0.79	0.84	0.86	0.8498
DenseNet	0.88	0.85	0.87	0.8598
PDCNet	0.91	0.92	0.91	0.9149

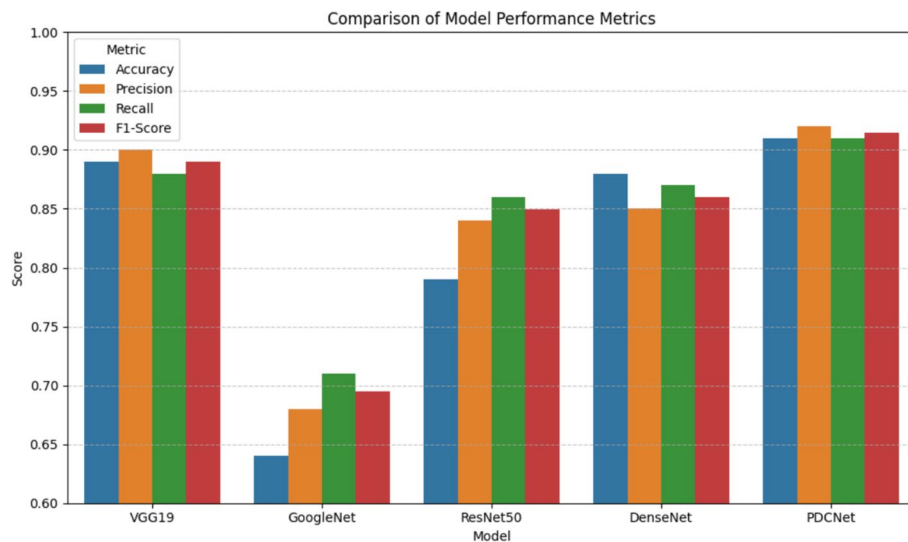


Fig. 12 Comparative Graph of The Proposed Method with Four State-Of-The-Art Methods

Table 13 Comparative analysis with other existing methods

Study / Approach	Dataset/Disease Classes	Method	Accuracy (%)
Mohantyt al. (2016) [3]	PlantVillage (38 classes)	AlexNet, GoogleNet	99.35
Toot al. (2019) [4]	PlantVillage (Bacteriaspot and Healthy)	Inception-V3	99.76
Chent al. (2020) [13]	PlantVillage (Bacteriaspot and Healthy)	ResNet50	98.90
Kumart al. (2021) [6]	PlantVillage (multiple classes, incl. bacterial spot)	DenseNet121	98.50
Our Work (PDCNet)	PlantVillage (Bacteriaspot and Healthy)	Segmentation + PDCNet	99.05

perform classification. In contrast, we provide a disease-level comparison where classification is not based on plant type (for example, "Bacteriaspot" vs. "Healthy"). This analysis demonstrated that our three-phase PDCNet approach achieves competitive performance (91% accuracy). This disease-level perspective is essential for real-world applications where disease characteristics often overlap across plant species.

In addition, we also conducted test on the PlantVillage dataset. Table 13 displays a comparison of the proposed model with other methods that have already been used to run tests on the plantvillage dataset. Our model, PDCNet, has a little lower accuracy (99.05%) than some of the results that have already been reported using PlantVillage data (for example, Toot al. (2019) got 99.76%). However, our model operates efficiently, and its results more accurately reflect its adaptability and generalization to real-world conditions, making it highly suitable for use in actual agricultural contexts.

Comparison with transfer learning technique

For image categorization, developing DNN models from scratch is a costly and time-consuming endeavor. Moreover, the training of deep learning models heavily depends on the availability of a substantial dataset, becoming a challenge when dealing with limited datasets [47]. Transfer learning, a technique used in deep

Table 14 No. of features extracted by pretrained cnn models

Name	Input Image Size	Output Features
VGG19	224×224×3	4096
GoogLeNet	224×224×3	1024
ResNet50	224×224×3	2048

Table 15 Comparison of the proposed method with pretrained cnn models on segmented images of pdsd dataset

Models	Dataset Division (Training–Testing)					
	70–30%			80–20%		
	Accuracy (%)	Deviation	Time (in sec.)	Accuracy (%)	Deviation	Time (in sec.)
VGG19	83.3%	±2.74	1.1×10 ⁴	83.9%	±2.50	1.2×10 ⁴
GoogleNet	83.8%	±3.85	8.5×10 ³	85.2%	±1.87	8.6×10 ³
ResNet50	84.2%	±2.52	6.6×10 ³	86.7%	±1.45	6.8×10 ³
PDCNet	88.7%	±1.96	9.2×10 ³	89.9%	±1.25	9.6×10 ³

learning, can effectively address these challenges. Pre-trained CNN models can be utilized for transfer learning that is specifically designed for one task but can transfer information to new domains. These pre-trained models can be used for making predictions, extracting features, and fine-tuning [48]. We employed widely recognized pre-trained models, namely ResNet50, VGG19, and GoogLeNet, as fixed feature extractors in order to evaluate the performance of transfer learning in comparison to our proposed model. The main concept is to utilize these pre-trained models without their final classification layer to extract features from simplified segmented images of PDSd dataset. These features are subsequently employed as input for multi-class SVM classifiers, enabling the categorization of plant images into twelve distinct classes.

The all models used in this paper needs input size to be 224×224×3. Therefore, all of the input images from the PDSd dataset of simplified segmented images have been resized to this dimension. In order to obtain the features representations of the training and test images, we modified the global pooling layer located at the end of the network. The global pooling layer aggregates the input data over all spatial locations, resulting in a diverse set of features. The Table 14 displays the number of features extracted by all the aforementioned CNN models. The detail description of the aforementioned CNN models beyond the limitations of this study.

Performance of transfer learning by using pre-trained VGG19, GoogleNet, ResNet-50 is compared with proposed PDCNet models on PDSd dataset, and it is highlighted in Table 15. The simplified images of PDSd dataset are used to train and identification of class and category of disease. The dataset is divided into training data and testing data. To analysis the effectiveness of models, dataset is partition into two sets of training–testing data: (i) 70%–30%; and (iii) 80%–20%. The classification accuracy was determined by computing the average accuracy over 10 trials. According to the results from Table 15, The proposed PDCNet achieved highest accuracy (88.7%

with training–testing partition of 70%–30% and 89.9% with training–testing partition of 80–20%) among three of the trained models. However, Proposed model takes more time (1.2×10^4 with training–testing partition of 80–20%) to complete the training because it is trained from scratch. Among all of them, Resnet50 takes less training time (6.6×10^3 with training–testing partition of 70–30%).

System performance recommendation

Once a specific disease is identified, the integrated solution recommender module provides optimal management strategies. This module leverages a database of predefined solutions to generate tailored recommendations, which are then delivered to the farmer. This approach facilitates effective disease prevention and management, aiming to minimize associated costs for agricultural producers (Fig. 13).

The data pertaining to diagnosed diseases in plant leaves will be presented in the assigned classification column (see Fig. 14), accompanied by the corresponding percentage indicating the extent of the affected area in case of infection. The description box will advise the user on the best course of action to take in the event of a leaf infection while taking the specific disease type into account.

Conclusion

Precision agriculture relies heavily on the segmentation methods employed to identify and remove infected leaves from plant images, as well as the deep learning model used to train the dataset. This paper provides a recommendation system for precision farming. It includes the recent work in the field of disease classification with deep learning, the proposed working model of PDCNet, experimental results, and discussion. Some of the key findings are as follows:

Key findings and advantages

The experimental results show that the proposed PDCNet model, when integrated with its Min–Max Hue Histogram-based segmentation approach, consistently outperforms several state-of-the-art CNN architectures (e.g., VGG19, ResNet50, GoogleNet, and

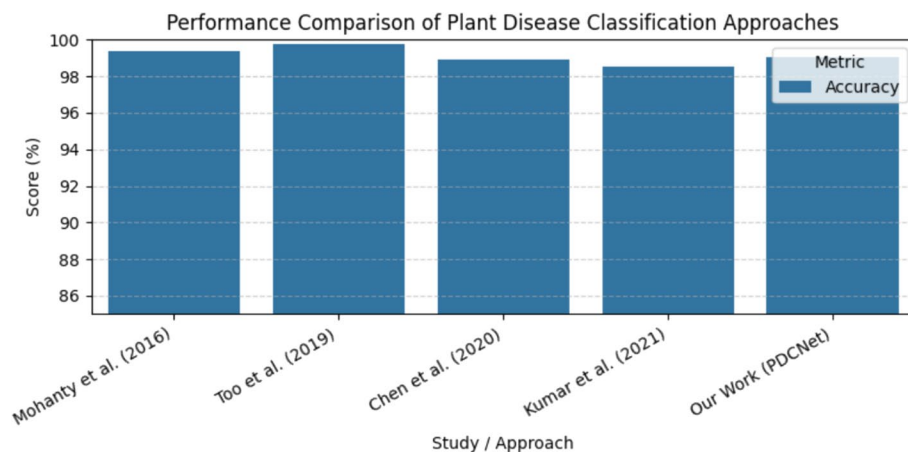


Fig. 13 Comparative Graph of The Proposed Method with other existing Methods

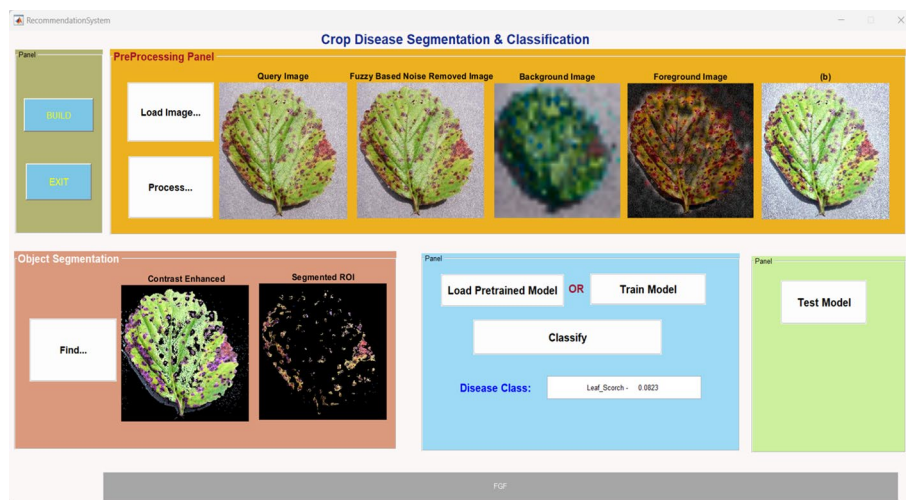


Fig. 14 Shows detection of leaf image with Leaf_Scorch disease with percentage

DenseNet) in plant disease classification on small datasets. The PDCNet is notably achieved an average accuracy of 91%, representing a 2–3% improvement over other competitive methods. This superior performance, particularly on small, heterogeneous datasets and without requiring data augmentation or transfer learning, is attributed to its the three-phase framework-image segmentation, deep learning classification, and disease recovery. Furthermore, our approach simplifies the classification task by isolating the region of interest before training which significantly reduces overall computational complexity and training time. These findings collectively indicate that the proposed methodology provides a robust and cost-effective solution for precision agriculture applications.

Limitations and future work

On small-scale plant disease datasets, the proposed PDCNet approach exhibits promising results; however, there are a number of issues that must be resolved. First, by eliminating background noise, our segmentation technique improves accuracy, however, it may struggle to handle plant images with complex textures or lighting conditions that are not included in the PDSD dataset. Second, because the model is trained from scratch, it results in longer training times compared to some transfer learning-based techniques. Third, our study utilized a limited subset of the PlantVillage dataset. Generalizing to a real-world images captured under divers weather conditions and with various capture devices would present further challenges.

Future plans will involve: (1) expanding the PDSD dataset and adding more data sources; (2) testing the model's robustness on a range of real-world photos; and (3) investigating lightweight model architectures that enables quicker training without compromising accuracy.

Author contributions

All authors contributed to the study conception and design. Material preparation, data collection and analysis were performed by Vijay Kumar Trivedi, Piyush Kumar Shukla and Anjana Pandey. The first draft of the manuscript was written by Vijay Kumar Trivedi and Noha Alduaiji and all other all other authors commented on previous versions of the manuscript. Noha Alduaiji, Shreyas read, dit and approved the final manuscript.

Funding

Open access funding provided by Manipal Academy of Higher Education, Manipal. No fund has been used for this research work.

Data availability

PDSD data is freely available online at <https://github.com/vkt911/PDSD-data-set>.

Declarations

Ethics approval and consent to participate

The submitted work is original and have not been published elsewhere in any form or language. Data is collected from standard, open-access PlantVillage dataset.

Competing interests

The authors declare no competing interests.

Received: 27 June 2024 Accepted: 3 August 2025

Published online: 29 November 2025

References

- Alston JM, Pardey PG. Agriculture in the global economy. *J Econ Perspect*. 2014;28:121–46.
- Li L, Zhang S, Wang B. Plant disease detection and classification by deep learning—a review. *IEEE Access*. 2021;9:56683–98.
- Mohanty SP, Hughes DP, Salathé M. Using deep learning for image-based plant disease detection. *Front Plant Sci*. 2016;7:1419. <https://doi.org/10.3389/fpls.2016.01419>.
- Too EC, Yujian L, Njuki S, Yingchun L. A comparative study of fine-tuning deep learning models for plant disease identification. *Comput Electron Agric*. 2019;161:272–9. <https://doi.org/10.1016/j.compag.2018.03.032>.
- Karlekar A, Seal A. SoyNet: soybean leaf diseases classification. *Comput Electron Agric*. 2020;172: 105342. <https://doi.org/10.1016/j.compag.2020.105342>.
- Kumar R, Singh SK, Singh SK, Vats M. Plant disease detection using deep learning and convolutional neural network. *Indian J Agric Sci*. 2021;91(9):1301–8.
- Panchal AV, Patel SC, Bagyalakshmi K, Kumar P, Khan IR, Soni M. Image-based plant diseases detection using deep learning. *Mater Today Proc*. 2021. <https://doi.org/10.1016/j.matpr.2021.07.281>.
- nkvetchakul A, Smith J, Patel R. A loss-fused, resilient CNN model for plant disease classification. *J Mach Learn Agric*. 2021;10(3):234–45.
- Pradeep, A.R.; Hoque, A.M.; Kabir, M.M.; Rahman, M.S.; Mridha, M.F. Plant Disease Identification from Leaf Images using Deep CNN's EfficientNet. In Proceedings of the 2022 International Conference on Decision Aid Sciences and Applications (DASA), Chiangrai, Thailand, 23–25 March 2022; pp. 523–527.
- Ahmed T, Khan AM, Raza M. A two-phase CNN architecture for the detection of rice crop diseases. *J Agric Eng Technol*. 2023;56(4):128–42.
- Narayanan KL, Krishnan RS, Robinson Julie YHG, Vimal S, Saravanan V, Kaliappan M. Banana plant disease classification using hybrid convolutional neural network. *Comput Intell Neurosci*. 2022;2022:9153699.
- Astani M, Hasheminejad M, Vaghefi M. A diverse ensemble classifier for tomato disease recognition. *Comput Electron Agric*. 2022;198: 107054.
- Chen J, Chen J, Zhang D, Sun Y, Nanekaran YA. Using deep transfer learning for image-based plant disease identification. *Comput Electron Agric*. 2020;173: 105393. <https://doi.org/10.1016/j.compag.2020.105393>.
- Jadhav SB, Udupi VR, Patil SB. Identification of plant diseases using convolutional neural networks. *Int J Inf Technol*. 2020. <https://doi.org/10.1007/s41870-020-00437-5>.
- Mohsin Kabir M, Quwsar Ohi A, Mridha MF. A multi-plant disease diagnosis method using convolutional neural network. *ArXiv*. 2020. https://doi.org/10.1007/978-981-33-6424-0_7.
- Krizhevsky A, Sutskever I, Hinton GE. Imagenet classification with deep convolutional neural networks. *Commun ACM*. 2017;60(6):84–90. <https://doi.org/10.1145/3065386>.
- Szegedy, C.; Liu, W.; Jia, Y.; Sermanet, P.; Reed, S.; Anguelov, D.; Erhan, D.; Vanhoucke, V.; Rabinovich, A. Going Deeper with Convolutions. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Boston, MA, USA, 7–12; pp. 1–9.
- Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition. *Comput Vision Pattern Recogn*. 2014. <https://doi.org/10.4855/arXiv.1409.1556>.
- He, K.; Zhang, X.; Ren, S.; Sun, J., 2016. Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778.
- Rusakovskiy O, Deng J, Su H, Krause J, Sathesh S, Ma S, Hung Z, Karpathy A, Khosla A, Bernstein M, Berg AC, Fei-Fei L. Imagenet largescale visual recognition challenge. *Inter J Comput Vis*. 2014;115:211–52.
- Huang W, Guan Q, Luo J, Zhang J, Zhao J, Liang D, Huang L, Zhang D. New optimized spectral indices for identifying and monitoring winter wheat diseases. *IEEE J Sel Top Appl Earth Observ Remote Sens*. 2014;7(6):2516–24.
- LeCun Y, Bengio Y, Hinton G. Deep learning. *Nature*. 2015;521(7553):436.
- Chollet, F., 2017. Xception: deep learning with depthwise separable convolutions. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1251–1258.
- Enkvetchakul P, Surinta O. Effective data augmentation and training techniques for improving deep learning in plant leaf disease recognition. *Appl Sci Eng Prog*. 2022;15:3810.

25. Arsenovic M, Karenovic M, Sledojcic S, Andarla A, Stefanovic D. Solving current limitations of deep learning based approach for plant disease detection. *Symmetry*. 2019;11:939.
26. Abbas A, Jain S, Gour M, Vankudothu S. Tomato plant disease detection using transfer learning with C-GAN synthetic images. *Comput Electron Agric*. 2021;187: 106279.
27. Anh, P.T.; Duc, H.T.M. A Benchmark of Deep Learning Models for Multi-leaf Diseases for Edge Devices. In Proceedings of the 2021 International Conference on Advanced Technologies for Communications (ATC), Ho Chi Minh City, Vietnam, 14–16 October 2021; pp. 318–323.
28. Cogswell M, Ahmed F, Girshick R, Zitnick L, Batra D. Reducing overfitting in deep networks by decorrelating representations. *ArXiv Preprint*. 2015. <https://doi.org/10.48550/arXiv.1511.06068>.
29. Thakur P, Khanna P, Sheorey T, Ojha A. Explainable vision transformernabled convolutional neural network for plant disease identification: PlantXViT. *ArXiv Preprint*. 2022. <https://doi.org/10.48550/arXiv.2207.07919>.
30. Quan S, Wang J, Jia Z, Yang M, Xu Q. MS-net: a novel lightweight and precise model for plant disease identification. *Front Plant Sci*. 2023;14: 1276728. <https://doi.org/10.3389/fpls.2023.1276728>.
31. Zhao X, Li K, Li Y, Ma J, Zhang L. Identification method of vegetable diseases based on transfer learning and attention mechanism. *Comput Electron Agric*. 2022;193:106763.
32. M. R. Ahmed et al., "Towards Automated Detection of Tomato Leaf Diseases," 2024 6th International Conference on Electrical Engineering and Information & Communication Technology (ICEEICT), Dhaka, Bangladesh, 2024, pp. 387–392, <https://doi.org/10.1109/ICEEICT62016.2024.10534559>.
33. Yang S, Zhang L, Lin J, Cernava T, Cai J, Pan R, Liu J, Wen X, Chen X, Zhang X. LSGnet: a lightweight convolutional neural network model for tomato disease identification. *Crop Prot*. 2024;182: 106715. <https://doi.org/10.1016/j.cropro.2024.106715>.
34. Wang Y, Yin Y, Li Y, Qu T, Guo Z, Peng M, Jia S, Wang Q, Zhang W, Li F. Classification of plant leaf disease recognition based on self-supervised learning. *Agronomy*. 2024;14:500. <https://doi.org/10.3390/agronomy14030500>.
35. https://www.tensorflow.org/datasets/catalog/plant_village
36. Fitri Utaminingsrum, Keiichi Uchimura and Gou Koutaki, "High Density Impulse Noise Removal based on Linear Mean-Median Filter", The 19th Korea-Japan Joint Workshop on Frontiers of Computer Vision, 2013.
37. Hamed Vahdat Nejad, Hameed Reza Pourreza, and Hasanbrahimi, A Novel Fuzzy Technique for Image Noise Reduction, *World Academy of Science, Engineering and Technology* 21 2008.
38. "CIE 1976 L*a*b* colour space [ilv]". Archived from the original on 2019-12-28. Retrieved 2020-12-12.
39. He K, Sun J, Tang X. Guided image filtering. *IEEE Trans Pattern Anal Mach Intell*. 2015;35(6):397–1409.
40. Mrunalini R. Badnakhe, P. R. Deshmukh 2012. Infected leave Analysis and Comparison by Otsu's Threshold and k-Means Clustering. *IJARCSASE*. 2, 1-3.
41. Mokhtar, Nurhayati & Harun, Nor & Mashor, Mohd & H, Roseline & Mustafa, Nazahah & Adollah, Robiyanti & Hashim, Adilah & Mohd Nasir, Nashrul. 2009. *Image enhancement Techniques Using Local, Global, Bright, Dark and Partial Contrast Stretching For Acute Leukemia Images*. *Lecture Notes in Engineering and Computer Science*. 2176.
42. Trivedi VK, Shukla PK, Pandey A. Automatic segmentation of plant leaves disease using min-max hue histogram and k-mean clustering. *Multimed Tools Appl*. 2022;81:20201–28. <https://doi.org/10.1007/s11042-022-12518-7>.
43. Saravanan G, Yamuna G, Nandhini S. Real time implementation of RGB to HSV/HSI/HSL and its reverse color space models. *Int Conf Commun Signal Process (ICCSPP)*. 2016;2016:0462–6. <https://doi.org/10.1109/ICCSPP.2016.7754179>.
44. Trivedi VK, Shukla PK, Dutta PK. K-mean and HSV model-based segmentation of unhealthy plant leaves and classification using machine learning approach. *IET Conf Proc*. 2021. <https://doi.org/10.1049/icp.2021.0953>.
45. Diederik P. Kingma, Jimmy Ba, Adam: A Method for Stochastic Optimization, a conference paper at the 3rd International Conference for Learning Representations <https://doi.org/10.48550/arXiv.1412.6980>
46. Reddy SVG, Thammi Reddy K, Valli Kumari V. Optimization of deep learning using various optimizers, loss functions and dropout. *Int J Innov Technol Explorng*. 2018;8(25):272–9.
47. Mohameth F, Bingcai C, Sada KA. Plant disease detection with deep learning and feature extraction using Plant Village. *J Comput Commun*. 2020. <https://doi.org/10.4236/jcc.2020.86002>.
48. Nguyen T-H, Nguyen T-N, Ngo B-V. A VGG-19 model with transfer learning and image segmentation for classification of tomato leaf disease. *AgriEngineering*. 2022;4:871–87. <https://doi.org/10.3390/agriengineering4040056>.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.